Penetration Testing Report

Client: Demo Client

Target: aspnet.testsparker.com

Assessment Type: Penetration Testing (Demo)

Assessment Date: 8:25 PM - 12/22/24

Prepared By: Vulnpire

Disclaimer

This report is a demonstration of a penetration test conducted for assessment purposes. **The findings, analysis, and recommendations are tailored for this specific demo and do not fully reflect my expertise, methodology, or the depth of my skills.** For a comprehensive penetration test, please consider the scope, tools, and time required for a full engagement.

Table of Contents

---

1. Executive Summary

This report outlines the findings from a penetration test conducted on the target system, aspnet.testsparker.com. The purpose of this assessment was to identify potential vulnerabilities and provide recommendations for improving the security posture of the target application.

Key Findings:

- [High/Medium/Low Severity] Vulnerabilities identified.

Critical Vulnerabilities:

- **Broken Access Control (Path Traversal):** Exploitable at Help.aspx via the item parameter, allowing unauthorized access to sensitive server files.
- **SQL Injection:**
    - POST-based injection at Converter.aspx via the JSON body, leading to sensitive database data extraction.
    - GET-based injection at Products.aspx (pId parameter), enabling partial database enumeration.

- **Stored Cross-Site Scripting (XSS):** Found in the Guestbook.aspx comment section, allowing persistent execution of malicious scripts.

Medium-Severity Vulnerabilities:

- **Reflected XSS:** Detected in the About.aspx endpoint via the hello parameter.
- **Insecure CAPTCHA Mechanism:** Weak CAPTCHA at Login.aspx permits brute-force attacks through OCR bypass.
- **Lack of Brute-Force Protection:** Login endpoint (Login.aspx) allows unlimited login attempts, increasing the risk of account compromise.
- **Clickjacking:** Vulnerable due to missing frame embedding protection.

Low-Severity Vulnerabilities:

- **Open Redirect:** Found at redirect.aspx (site parameter), redirecting users to untrusted locations.
- **Broken Link Hijacking:** Social media link vulnerability due to unclaimed or inactive accounts.

Recommendations:

- Implement security measures to address identified vulnerabilities.
- Regularly conduct security assessments to maintain a strong security posture.

---

2. Scope and Objectives

Scope:

- Target Application: aspnet.testsparker.com
- Testing Type: External black-box penetration test.

Objectives:

- Identify vulnerabilities in the web application.
- Assess the risk and potential impact of identified issues.
- Provide actionable recommendations for remediation.

2. Target Details

- Technology Stack: Microsoft IIS 8.5, ASP.NET 4.0.30319
- Database: Microsoft SQL Server 2014
- Operating System: Windows Server 2012 R2 or 8.1

---

3. Methodology

The following methodology was used to conduct the penetration test:

1. Reconnaissance:
    - Passive and active information gathering.
    - Identification of subdomains, IP addresses, and technology stack.
    - Steps Taken:
        - I usually start my reconnaissance by trying to get origin IP addresses using one of my tools sXtract, a process that bypasses the Web Application Firewall (WAF).
        - To avoid WAF bans, I used Tor to change my IP address at regular intervals: `screen -dmS tornet bash -c "sudo tornet --interval 5 --count 2"`
        - Fuzzing was performed using dirsearch to identify hidden directories and files:

```
dirsearch -u http://aspnet.testsparker.com -t 100 -r -R 2
--random-agent -e
php,aspx,asp,cfm,jsp,jspx,log,bak,old,backup,git,zip,tar.gz,tar,sql,env,i
ni,swp,tmp,json,xml,conf --proxy socks5://127.0.0.1:9050 --full-url
--no-color -x 429,404,403,401,500,503 --retries=3 --timeout 4 --delay
0.5
```

2. Scanning and Enumeration:
    - Automated vulnerability scanning and exploitation using tools like Nmap, OWASP Pentesting kit, and SQLMap.
    - Manual enumeration of endpoints and services.
3. Exploitation:
    - Attempted exploitation of identified vulnerabilities.
    - Verification of findings to confirm validity.
4. Post-Exploitation:
    - Assessment of potential impacts (data exposure, privilege escalation, etc.).
5. Reporting:
    - Documentation of findings, including proof of concept (PoC) where applicable.
    - Recommendations for remediation.

---

4. Findings and Analysis

4.1 High/Critical-Severity Issues

A1: Broken Access Control

4.1.1 Path Traversal

- Description: The application is vulnerable to a path traversal attack at http://aspnet.testsparker.com/Help.aspx. By manipulating the item parameter, an attacker can access sensitive files on the server.
- Impact: This vulnerability allows unauthorized access to server files, potentially exposing sensitive information such as configuration files, credentials, or system details.
- Proof of Concept (PoC):
  URL:
  http://aspnet.testsparker.com/Help.aspx?item=../../../windows/system32/drivers/etc/hosts
  Behavior: Accesses the server's hosts file, confirming the ability to read arbitrary files.
- Recommendation:
  1. Validate and sanitize the item parameter to ensure only valid input is accepted.
  2. Implement allowlist-based validation for file access.
  3. Restrict access to sensitive directories and files at the server level.

A3: Injection

4.1.2 SQL Injection

- Endpoint: http://aspnet.testsparker.com/Converter.aspx
- Payload: SQL injection via JSON POST body.

Command Used:

```
sqlmap -u "http://aspnet.testsparker.com/ConverterResponse.aspx" --data
'{"btcAmount":"1"}' --dbms="Microsoft SQL Server"  --batch --level=5 --risk=3 --dbs --dump
--proxy socks5://127.0.0.1:9050 --random-agent
```

Database Information:

- Web Server OS: Windows 2012 R2 or 8.1.
- Web Application Technology: Microsoft IIS 8.5, ASP.NET 4.0.30319.
- Back-end Database Management System: Microsoft SQL Server 2014.

Available Databases:

- ASPState
- master
- model
- msdb
- tempdb
- testsparker

Targeted Database:

- Current database: testsparker.

Tables in testsparker:

- aspnet_Applications
- aspnet_Membership
- aspnet_Paths
- aspnet_PersonalizationAllUsers
- aspnet_PersonalizationPerUser

- aspnet_Profile
- aspnet_Roles
- aspnet_SchemaVersions
- aspnet_Users
- aspnet_UsersInRoles
- aspnet_WebEvent_Events
- tblBlog
- tblBtcValues
- tblComments
- tblEmployees
- tblProducts
- vw_aspnet_*

Dumped Data:

- Tables such as aspnet_Applications, aspnet_Membership, tblComments, tblEmployees, and tblProducts were successfully dumped into CSV files.

Notable Table Details:

- tblEmployees contains records with employee details such as employeeName, employeeId, employeeImage, etc.
- tblProducts contains 5 entries, likely related to products or services offered by the application.
- Sensitive information (e.g., aspnet_Membership, aspnet_Profile) including user data, though some tables appear empty.

Next Steps:

- Examine the dumped CSV files for sensitive data such as user credentials, personally identifiable information (PII), or other critical business information.

Further Exploitation:

- Use the --dump flag with specific tables of interest (e.g., aspnet_Membership or tblEmployees) to extract additional details.
- Use sqlmap's --os-shell option to attempt gaining an interactive shell on the underlying operating system.

Mitigation Recommendations:

- Validate and sanitize user inputs (e.g., JSON POST body parameters).
- Implement parameterized queries or stored procedures to prevent SQL injection.
- Enable database logging to detect unusual activity.

4.1.3 SQL Injection (GET Method)

Endpoint:

http://aspnet.testsparker.com/Products.aspx?pId=4

Description:
A SQL injection vulnerability was identified in the pId parameter of the Products.aspx endpoint. The vulnerability allows attackers to manipulate SQL queries executed by the database, potentially leading to unauthorized data access or other exploits.

Command Used:

```
sqlmap -u "http://aspnet.testsparker.com/Products.aspx?pId=4" --dbs
--tamper="space2comment" --random-agent --hostname -D testsparker -p "pId"
--tables --tamper=between,space2dash --dump --batch --proxy
socks5://127.0.0.1:9050 --current-user --level=2 --risk=2 --flush-session
```

Findings:

- SQL injection was confirmed, but attempts to retrieve detailed database information were unsuccessful.
- Further exploitation was not pursued, as this assessment was conducted solely for demonstration purposes.

Impact:

- Exploiting this vulnerability could allow attackers to access sensitive database information, including user details, application data, and potentially sensitive configurations.

Recommendation:

1. Input Validation and Sanitization: Ensure all user inputs are validated and sanitized to prevent malicious SQL commands.
2. Use Parameterized Queries: Replace dynamic SQL queries with parameterized ones or stored procedures to eliminate injection risks.
3. Error Handling: Avoid exposing database error messages to users to prevent information leakage.
4. Implement Security Mechanisms: Deploy Web Application Firewalls (WAFs) and monitoring tools to detect and block malicious activities.
5. Conduct Regular Security Audits: Perform routine penetration testing to identify and address vulnerabilities promptly.

A7: Cross-Site Scripting (XSS)

4.1.4 Stored Cross-Site Scripting

Endpoint:

http://aspnet.testsparker.com/Guestbook.aspx

Description:

The application is vulnerable to a Stored XSS vulnerability. By injecting malicious JavaScript code into the comment section of the Guestbook feature, the script is saved and executed whenever the page or comment is viewed.

Proof of Concept:

Payload:

- <script>prompt(1)</script>

Behavior:

- When the payload is submitted in the comment section, it is stored in the application’s database.
- Upon visiting the Guestbook page, the payload is executed, triggering a prompt(1) dialog box in the victim's browser.

Impact:

Stored XSS vulnerabilities have significant security implications, including:

- Session Hijacking: Stealing cookies or session tokens to impersonate users.
- Credential Theft: Redirecting users to phishing pages or stealing credentials via keylogging.
- Malware Distribution: Injecting malicious code to download malware onto victim devices.
- Privilege Escalation: Targeting administrative users to gain further control of the application.

Recommendations

1. Input Validation and Sanitization:
    - Validate all user inputs on both client and server sides.
    - Strip or encode special characters like <, >, &, and ' from user input.
2. Output Encoding:
    - Ensure all data retrieved from the database is properly encoded before rendering on the web page.
    - Use libraries such as Microsoft.AntiXss for encoding in ASP.NET applications.
3. Content Security Policy (CSP):
    - Implement a strict Content Security Policy to restrict the execution of inline scripts and only allow trusted sources.
4. Escaping Dynamic Content:
    - Escape all untrusted data before using it in HTML, JavaScript, or CSS.
5. Regular Security Audits:

- Perform routine penetration tests to detect and remediate XSS vulnerabilities.

4.2 Low/Medium-Severity Issues

A4: Insecure Design

4.2.1 Open Redirect

Description: The application contains an open redirect vulnerability at http://aspnet.testsparker.com/redirect.aspx. By modifying the site parameter, an attacker can redirect users to a malicious website.

Impact: This vulnerability can be exploited for:

- Phishing attacks
- Stealing cookies
- Redirecting users to malicious sites
- Bypassing certain security controls

Proof of Concept (PoC):

- URL: http://aspnet.testsparker.com/redirect.aspx?site=evil.com
- Behavior: Redirects the user to evil.com.

Recommendation: Validate and sanitize the site parameter to ensure only approved domains are allowed. Implement allowlist-based validation for redirects.

A7: Cross-Site Scripting (XSS)

4.2.2 Reflected XSS

Description: The application is vulnerable to reflected XSS at http://aspnet.testsparker.com/About.aspx by injecting JavaScript into the hello parameter.

Proof of Concept (PoC):

- URL: http://aspnet.testsparker.com/About.aspx?hello=%3Cscript%3Eprompt(1)%3C/script%3E
- Behavior: Executes the injected JavaScript, triggering a prompt(1) dialog.

Impact:

- Execution of malicious scripts in the user's browser
- Stealing cookies or session tokens
- Redirecting users to malicious sites

Recommendation: Sanitize user input and implement output encoding to prevent JavaScript execution. Use Content Security Policy (CSP) headers to mitigate XSS risks.

4.2.3 DOM-Based Cross-Site Scripting (DOM XSS)

Endpoint:

http://aspnet.testsparker.com/Pricings.aspx#alert(1)

Description:

The application is vulnerable to DOM-based Cross-Site Scripting (XSS) due to the unsafe use of user-controlled input (location.href.split("#")[1]) in JavaScript without proper validation or sanitization. The vulnerable code allows attackers to inject malicious scripts into the page, leading to arbitrary JavaScript execution.

Proof of Concept:

- Payload: http://aspnet.testsparker.com/Pricings.aspx#alert(1)
- Behavior: When the above payload is executed in a Chrome browser, it triggers a JavaScript alert(1) dialog box, confirming the vulnerability.
- Browser-Specific Note: The exploit does not work in Firefox but functions as intended in Chrome. This behavior may be due to differences in how browsers handle fragment identifiers or execute JavaScript.

Vulnerable Code:

```
var taintedVariable = location.href.split("#")[1];

setTimeout("var x=" + taintedVariable, 500);
```

Fixed Example with DOMPurify:

```
var taintedVariable = location.href.split("#")[1];

var sanitizedValue = DOMPurify.sanitize(taintedVariable);

console.log(sanitizedValue);
```

Impact:

DOM-based XSS can be exploited to:

- Steal session cookies or tokens.
- Execute arbitrary JavaScript in the context of the user's browser.

- Redirect users to malicious sites.
- Perform actions on behalf of authenticated users.

Recommendations:

1. Input Validation and Sanitization:
    - Sanitize and validate all client-side inputs, including the URL fragment, before processing them.
    - Use libraries like DOMPurify to sanitize untrusted data in JavaScript.
2. Output Encoding:
    - Encode user inputs before inserting them into the DOM.
3. Content Security Policy (CSP):
    - Implement a strict CSP to prevent execution of unauthorized scripts.
4. Browser Compatibility Testing:
    - Test the application across multiple browsers to identify and mitigate browser-specific security issues.
5. Security Audits:
    - Regularly perform security audits focusing on client-side vulnerabilities.

A5: Security Misconfiguration

4.2.4 Clickjacking

Description: The application is vulnerable to clickjacking, allowing attackers to embed the application in an iframe to trick users into performing unintended actions.

Proof of Concept (PoC):

- Script: Clickjacking PoC Script

Impact:

- Unauthorized actions performed by the user
- Potential for phishing attacks or session hijacking

Recommendation: Implement the X-Frame-Options header with the value DENY or SAMEORIGIN. Use the Content-Security-Policy header with the frame-ancestors directive to control frame embedding.

A7 - Identification and Authentication Failures

4.2.5 Broken Link Hijacking

- URL: http://*.com/testinvicti
    - Issue: The linked social media account does not exist, making it susceptible to hijacking by malicious actors.
- Recommendation: Remove the broken link or ensure the account is claimed and actively managed.
- Unvalidated Input:
    - URL: http://aspnet.testsparker.com/About.aspx?hello=visitor
    - Issue: The hello parameter accepts unvalidated input, potentially leading to injection vulnerabilities.
- Recommendation: Validate and sanitize all user inputs to prevent abuse or exploitation.

A7: Identification and Authentication Failures

4.2.6 Lack of Brute-Force Protection

The application does not implement protection mechanisms against brute-force attacks on the login endpoint.

Endpoint:

POST /administrator/Login.aspx?r=%2fDashboard%2f

Description: The login functionality at the specified endpoint allows repeated login attempts without any restrictions or mechanisms to prevent brute-force attacks. This includes the absence of features such as:

- Account lockout after a defined number of failed attempts.
- CAPTCHA or similar verification mechanisms.
- Rate limiting to restrict repeated requests.

Potential Impact:
While this issue may not currently be exploited, it represents a poor security practice that could lead to the following:

- Unauthorized access to user accounts through credential stuffing or brute-force attacks.
- Increased likelihood of successful exploitation when combined with leaked credentials or weak passwords.
- Potential abuse by attackers to test login credentials at scale, leading to system resource exhaustion or denial-of-service conditions.

Recommendation:

1. Implement Rate Limiting: Restrict the number of login attempts per user or IP address within a specified time frame.
2. Introduce Account Lockout: Temporarily lock accounts after a defined number of failed login attempts.
3. Enable CAPTCHA: Add CAPTCHA functionality to the login page to prevent automated attacks.
4. Monitor and Alert: Log failed login attempts and generate alerts for anomalous activity patterns.

5. Educate Users: Encourage the use of strong, unique passwords and implement multi-factor authentication (MFA) to further mitigate risks.

A6: Vulnerable and Outdated Components

4.2.7 Insecure CAPTCHA Mechanism Permits Brute-Force Attacks

Description: The login functionality at http://aspnet.testsparker.com/administrator/Login.aspx has a weak CAPTCHA mechanism that can be bypassed, allowing attackers to attempt repeated login attempts without restrictions.

PoC (Demo):
Using an automated script, I was able to bypass the CAPTCHA by solving it dynamically with OCR and submitting multiple login attempts with a wordlist.

Impact:
Attackers can potentially gain unauthorized access to user accounts through credential stuffing or brute-force attacks, especially if weak passwords or leaked credentials are used.

Recommendation:

- Implement stronger CAPTCHA mechanisms such as Google reCAPTCHA v3 or **hCaptcha** to block automated login attempts effectively.
- Ensure CAPTCHA challenges are difficult to bypass using OCR or other automated methods.
- Consider implementing multi-factor authentication (MFA) to further secure login attempts.

6. Conclusion

This penetration testing assessment identified nine vulnerabilities on **aspnet.testsparker.com**, including three critical issues (SQL Injection, Path Traversal, and Stored XSS). Exploitation of these vulnerabilities could lead to unauthorized access to sensitive data, compromise of user accounts, and potential system control by attackers. Mitigation strategies have been provided to address these risks and strengthen the application's security posture. Immediate action is recommended to remediate critical issues and implement preventive measures.

This report is for **demonstration purposes only and does not reflect the full extent of my skills or methodologies.** For a comprehensive security assessment, additional time, resources, and scope would be required.

Thank you for the opportunity to demonstrate my approach to penetration testing.